Learning Rust Through a Portable Graph Loader, Edition 2

Architecture, traits, SDKs, functional Rust, and the By the Bay graph pipeline

Ownership, borrowing, data modeling, backend abstraction, SDK transports, and practical Rust design

Generated from the implementation in `src/main.rs`, `src/bin/load_graph.rs`, and `src/graph_loader.rs`

May 2026

Contents

Preface

This second edition teaches Rust through one concrete implementation: the By the Bay graph pipeline. The project has two executables and one shared library module. One executable scrapes and extracts meetup and conference data. The other executable loads that neutral graph into several database backends: FalkorDB, HelixDB, and SurrealDB.

The important part is not the databases. The important part is the Rust design: how data moves through the program, how ownership is preserved, where borrowing matters, how errors are propagated, how the command line maps to runtime configuration, and how backend-specific behavior is hidden behind a stable API.

The first edition focused heavily on ownership and the borrow checker. Edition 2 keeps that material, but broadens the view to architecture: module boundaries, data contracts, traits, SDK-vs-HTTP backend implementations, iterator-heavy functional style, tests, reuse, and the Rust features that appear in the real code.

The code discussed here lives mainly in:

```
src/main.rs
src/bin/load_graph.rs
src/graph_loader.rs
```

1. The Shape of the Program

The loader has two main layers.

The binary, `src/bin/load_graph.rs`, owns command-line parsing and input conversion. It reads either:

- a consolidated export, `--input-format export`
- Claude-style per-talk records, `--input-format talk-records`

Both input paths produce the same neutral value:

```
GraphData {
    nodes: Vec<GraphNode>,
    edges: Vec<GraphEdge>,
}
```

The library module, `src/graph_loader.rs`, owns database loading. Its public API is intentionally small:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()>
```

That tells you the whole contract:

- The caller owns the configuration.
- The caller owns the graph data.
- The loader borrows both.
- The loader either succeeds or returns an error.

This is a good Rust boundary. It is explicit, cheap to call, and difficult to misuse.

2. Command-Line Parsing With `clap`

The binary starts with a struct that describes the CLI:

```
#[derive(Debug, Parser)]
#[command(version, about = "Load By the Bay graph JSON into a graph database")]
struct Args {
    #[arg(short, long, default_value = "data/bythebay-graph.json")]
    input: PathBuf,

    #[arg(long, value_enum, default_value_t = InputFormat::Export)]
    input_format: InputFormat,

    #[arg(long, value_enum, default_value_t = GraphBackend::Falkor)]
    backend: GraphBackend,

    #[arg(long, default_value_t = 100)]
    batch_size: usize,
}
```

This is idiomatic Rust application code:

- `PathBuf` is an owned filesystem path.
- `usize` is the natural size type for counts and slice indexes.
- `InputFormat` and `GraphBackend` are enums, not strings.

Using enums matters. A stringly typed CLI lets invalid values drift deep into the program. A `ValueEnum` makes invalid values fail at parsing time.

```
#[derive(Debug, Clone, Copy, ValueEnum)]
enum InputFormat {
    Export,
    TalkRecords,
}
```

`Clone` and `Copy` are appropriate here because the enum is tiny and has no heap data. Passing it around by value is simpler than borrowing it.

3. Turning CLI Arguments Into Configuration

The CLI struct is specific to the binary. The library uses a separate config:

```rust
pub struct GraphLoadConfig {
    pub backend: GraphBackend,
    pub redis_url: String,
    pub helix_url: String,
    pub surreal_url: String,
    pub graph: String,
    pub replace: bool,
    pub batch_size: usize,
}
```

The conversion is implemented with `From`:

```rust
impl From<&Args> for GraphLoadConfig {
    fn from(args: &Args) -> Self {
        Self {
            backend: args.backend,
            redis_url: args.redis_url.clone(),
            helix_url: args.helix_url.clone(),
            surreal_url: args.surreal_url.clone(),
            graph: args.graph.clone(),
            replace: args.replace,
            batch_size: args.batch_size,
            // other fields omitted
        }
    }
}
```

This is one of the first places ownership matters.

`Args` owns its `String` fields. `GraphLoadConfig` also needs to own strings, because it may outlive the local `Args` borrow. Therefore, the conversion clones strings.

For small configuration strings, cloning is the right tradeoff. It avoids lifetime complexity and keeps the public config straightforward.

4. The Neutral Graph Model

The core data model is in `src/graph_loader.rs`:

```rust
#[derive(Debug, Clone, Default)]
pub struct GraphData {
    pub nodes: Vec<GraphNode>,
    pub edges: Vec<GraphEdge>,
}

#[derive(Debug, Clone)]
pub struct GraphNode {
    pub label: String,
    pub props: BTreeMap<String, GraphValue>,
}

#[derive(Debug, Clone)]
pub struct GraphEdge {
    pub from: String,
    pub to: String,
    pub relationship: String,
}
```

This is deliberately plain. A node has:

- one primary label
- a stable string ID in `props["id"]`
- a property map

An edge has:

- source node ID
- target node ID
- relationship name

The database-specific node IDs are not part of this model. That keeps the graph portable across backends.

Why `BTreeMap`?

Properties are stored in a `BTreeMap`:

```rust
pub props: BTreeMap<String, GraphValue>
```

A `HashMap` would also work. `BTreeMap` has one practical advantage here: deterministic iteration order. That makes generated Cypher, SurrealQL, and test output more stable.

Determinism is useful in loader code because query generation is easier to debug when the same input produces the same string order.

5. Modeling Property Values

The loader does not accept arbitrary JSON internally. It uses a small enum:

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum GraphValue {
    String(String),
    StringArray(Vec<String>),
}
```

This is a key design choice. The export data may start as JSON, but the loader supports only the value types it knows how to write safely across all backends.

That buys three things:

- each backend writer handles a finite set of cases
- unsupported values are converted at the importer boundary
- tests can compare values directly with `PartialEq`

The implementation also defines convenient conversions:

```
impl From<String> for GraphValue {
    fn from(value: String) -> Self {
        Self::String(value)
    }
}

impl From<&str> for GraphValue {
    fn from(value: &str) -> Self {
        Self::String(value.to_string())
    }
}
```

This lets calling code write:

```
("name", person.name.clone().into())
```

instead of:

```
("name", GraphValue::String(person.name.clone()))
```

6. Borrowing for Required Node IDs

Every node must have a string `id` property. The method that enforces this is:

```
impl GraphNode {
    fn id(&self) -> Result<&str> {
        match self.props.get("id") {
            Some(GraphValue::String(id)) => Ok(id),
            _ => bail!("all graph nodes must include a string id property"),
        }
    }
}
```

This is an excellent borrow-checker example.

The node owns the `String` inside `GraphValue::String`. The `id` method does not clone that string. It returns `&str`, a borrowed view into the owned string.

The returned reference cannot outlive `self`. Rust enforces that automatically. That is why this signature is safe:

```
fn id(&self) -> Result<&str>
```

The caller can use the borrowed ID while it is still working with the borrowed node, but cannot stash it somewhere longer-lived without copying it.

This keeps query generation cheap:

```
cypher_string(node.id()?)
```

No allocation is needed just to read the ID.

7. Error Handling With `anyhow`

The loader is an application-level tool, not a reusable low-level crate. It uses `anyhow::Result`:

```
use anyhow::{Context, Result, bail};
```

`bail!` returns an error immediately:

```
bail!("all graph nodes must include a string id property")
```

`Context` adds useful information to lower-level errors:

```
let graph_json = fs::read_to_string(input)
    .with_context(|| format!("failed to read {}", input.display()))?;
```

The `?` operator is central. It means:

- if the operation succeeds, unwrap the value
- if it fails, return the error from the current function

This keeps fallible code linear and readable.

8. Reading the Consolidated Export

The default importer reads one JSON file into typed structs:

```rust
fn read_export_graph(input: &PathBuf) -> Result<GraphData> {
    let graph_json = fs::read_to_string(input)
        .with_context(|| format!("failed to read {}", input.display()))?;
    let export: GraphExport = serde_json::from_str(&graph_json)
        .with_context(|| format!("failed to parse {}", input.display()))?;
    export.to_graph_data()
}
```

Notice the ownership path:

1. `fs::read_to_string` returns an owned `String`.
2. `serde_json::from_str` borrows that string while parsing.
3. `GraphExport` owns the parsed data.
4. `to_graph_data` borrows `GraphExport` and creates owned `GraphData`.

The export structs mirror the JSON:

```rust
#[derive(Debug, Deserialize)]
struct GraphExport {
    source_urls: Vec<String>,
    conferences: Vec<Conference>,
    meetups: Vec<MeetupGroup>,
    people: Vec<Person>,
    talks: Vec<Talk>,
    edges: Vec<Edge>,
}
```

These structs are private to the binary. That is a good boundary: the JSON format is an input concern, not a graph-loading concern.

9. Converting Export Records Into Nodes

Each export entity gets a small converter:

```
fn meetup_node(meetup: &MeetupGroup) -> GraphNode {
    let mut props = props([
        ("id", meetup.id.clone().into()),
        ("name", meetup.name.clone().into()),
        ("url", meetup.url.clone().into()),
    ]);
    insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
    GraphNode::new("Meetup", props)
}
```

This function borrows a `MeetupGroup` and returns an owned `GraphNode`.

Why clone? Because the graph node must own its property strings. The source `MeetupGroup` is only borrowed. Moving fields out of a borrowed struct is not allowed, and should not be allowed: the caller still owns the export.

`as_deref` is a subtle useful method:

```
meetup.timezone.as_deref()
```

It turns `Option<String>` borrowed through `&MeetupGroup` into `Option<&str>`. That matches the helper:

```
fn insert_optional(props: &mut BTreeMap<String, GraphValue>, key: &str, value: Option<&str>)
```

This avoids cloning optional strings unless they are actually present and non-empty.

10. Building Maps With Const Generics

The helper for property maps is:

```
fn props<const N: usize>(items: [(&str, GraphValue); N]) -> BTreeMap<String, GraphValue> {
    items
        .into_iter()
        .map(|(key, value)| (key.to_string(), value))
        .collect()
}
```

`const N: usize` lets the function accept arrays of any length while preserving static type information.

These all compile:

```
props([("id", "person:1".into())])

props([
    ("id", "meetup:1".into()),
    ("name", "Graph Night".into()),
    ("url", "https://example.test".into()),
])
```

Without const generics, you would likely accept a slice or vector. The array version is compact at call sites and allocation-free before the final map is collected.

11. Optional Properties and Mutable Borrows

Optional fields are inserted with:

```
fn insert_optional(props: &mut BTreeMap<String, GraphValue>, key: &str, value: Option<&str>) {
    if let Some(value) = value.filter(|value| !value.is_empty()) {
        props.insert(key.to_string(), value.into());
    }
}
```

This function takes `&mut BTreeMap<...>`.

That mutable borrow means the caller temporarily gives the helper exclusive access to the map. While the helper is running, no other code can read or write the same map. After the function returns, the borrow ends and the caller can use the map again.

This is the borrow checker protecting you from iterator invalidation and concurrent mutation bugs.

12. Alternate Importer: Per-Talk Records

The alternate importer accepts Claude-style talk record files:

```
#[derive(Debug, Deserialize)]
struct TalkRecord {
    nodes: Vec<TalkRecordNode>,
    edges: Vec<TalkRecordEdge>,
}
```

It supports flexible field names:

```
#[derive(Debug, Deserialize)]
struct TalkRecordNode {
    id: String,
    #[serde(alias = "type", alias = "kind")]
    label: String,
    #[serde(default)]
    properties: serde_json::Map<String, serde_json::Value>,
}
```

`serde(alias = "...")` is useful when ingesting adjacent formats. It lets the Rust code keep one field name, `label`, while accepting JSON that says `type` or `kind`.

The importer deduplicates into maps:

```
let mut nodes_by_id: BTreeMap<String, GraphNode> = BTreeMap::new();
let mut edges_by_key: BTreeMap<(String, String, String), GraphEdge> = BTreeMap::new();
```

Node identity is the stable string ID. Edge identity is the tuple:

```
(from, to, relationship)
```

This mirrors how graph loaders usually think about idempotence.

13. Moving Values Into Deduplication Maps

The edge deduplication code is:

```
edges_by_key
    .entry((
        edge.from.clone(),
        edge.to.clone(),
        edge.relationship.clone(),
    ))
    .or_insert_with(|| GraphEdge {
        from: edge.from,
        to: edge.to,
        relationship: edge.relationship,
    });
```

This is a compact ownership lesson.

The map key needs owned strings, so the key clones the fields. If the edge is not already present, `or_insert_with` moves the original strings into the stored `GraphEdge`.

Why not avoid the clones? You could restructure this code to clone less, but this version is simple and correct. For a loader dominated by database I/O, the small string clones are not the bottleneck.

Rust makes the cost visible, which lets you choose intentionally.

14. Converting JSON Values Safely

The alternate importer converts arbitrary JSON into the loader's smaller property enum:

```
fn json_graph_value(value: serde_json::Value) -> GraphValue {
    match value {
        serde_json::Value::String(value) => GraphValue::String(value),
        serde_json::Value::Array(values) => GraphValue::StringArray(
            values
                .into_iter()
                .filter_map(|value| match value {
                    serde_json::Value::String(value) if !value.is_empty() => Some(value),
                    _ => None,
                })
                .collect(),
        ),
        serde_json::Value::Null => GraphValue::String(String::new()),
        other => GraphValue::String(other.to_string()),
    }
}
```

This is boundary normalization. Inside the loader, backends do not need to deal with arbitrary JSON. They only handle `String` and `StringArray`.

The function takes `serde_json::Value` by value, not by reference. That means it can move strings out of the JSON without cloning:

```
serde_json::Value::String(value) => GraphValue::String(value)
```

If it took `&serde_json::Value`, this branch would need to clone the string.

15. Public API, Private Implementation

The public function stays small:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()> {
    match config.backend {
        GraphBackend::Falkor => FalkorLoader.load(config, graph),
        GraphBackend::HelixHttp => HelixHttpLoader.load(config, graph),
        GraphBackend::HelixRustSdk => HelixRustSdkLoader.load(config, graph),
        GraphBackend::Surrealdb => SurrealHttpLoader.load(config, graph),
        GraphBackend::SurrealdbRustSdk => SurrealRustSdkLoader.load(config, graph),
    }
}
```

The internal abstraction is a trait:

```
trait GraphLoader {
    fn load(&self, config: &GraphLoadConfig, graph: &GraphData) -> Result<()>;
}
```

The trait is private. That is deliberate.

The outside world does not need to know how backend dispatch works. It only needs `load_graph`. Keeping the trait private gives us freedom to change the internal design without breaking callers.

16. Why Not Return Trait Objects?

Another design would be:

```
fn backend_loader(config: &GraphLoadConfig) -> Box<dyn GraphLoader>
```

That is useful when the selected backend must be stored and reused as a runtime object. This loader does not need that. It dispatches once, performs the load, and exits.

The current match is simpler:

- no heap allocation for a boxed trait object
- no dynamic dispatch beyond the simple call
- easy to read
- easy to debug

Rust rewards choosing the simplest abstraction that fits the lifetime of the problem.

17. Backend-Specific Ownership

Each backend borrows the neutral graph and generates backend-specific commands.

For FalkorDB:

```
for node in &graph.nodes {
    let query = format!(
        "MERGE (n:{} {{id:{}}}) SET n += {}",
        falkor_labels(node),
        cypher_string(node.id()?),
        cypher_map(&node.props)
    );
    falkor_query(&mut connection, &config.graph, &query)?;
}
```

Important details:

- `&graph.nodes` iterates by shared reference.
- `node.id()?` borrows the ID.
- `cypher_map(&node.props)` borrows the property map.
- `query` is an owned string created for the database call.
- `connection` is borrowed mutably because Redis commands mutate connection state.

That last point is a common Rust pattern. Network clients and database connections often require `&mut self` because sending a command changes buffers, sequence state, or protocol state.

18. Escaping Strings for Cypher

Cypher strings are generated explicitly:

```rust
fn cypher_string(value: &str) -> String {
    let mut escaped = String::with_capacity(value.len() + 2);
    escaped.push('\'');
    for ch in value.chars() {
        match ch {
            '\\' => escaped.push_str("\\\\"),
            '\'' => escaped.push_str("\\'"),
            '\n' => escaped.push_str("\\n"),
            '\r' => escaped.push_str("\\r"),
            '\t' => escaped.push_str("\\t"),
            _ => escaped.push(ch),
        }
    }
    escaped.push('\'');
    escaped
}
```

The function borrows `&str` and returns a new escaped `String`.

`String::with_capacity(value.len() + 2)` is a small performance hint. The output will be at least two bytes longer because of the surrounding quotes. Escapes may make it longer, but this capacity avoids reallocating for the common case.

19. Backend Differences Shape the Code

The same graph model is written differently by each database.

FalkorDB is Cypher-like:

```
MERGE (n:Talk {id:'talk:1'}) SET n += {title:'Graph Loading'}
```

HelixDB uses dynamic graph mutations:

```
g().add_n(node.label.clone(), helix_sdk_properties(node))
```

SurrealDB uses records and edge tables:

```
CREATE type::record("talk", "talk:1") SET title = "Graph Loading";
RELATE (type::record("person", "person:1"))->presents->(type::record("talk", "talk:1"));
```

The abstraction works because all three can honor the same portable contract:

- stable string node IDs
- labels or tables derived from node labels
- relationships derived from edge names

20. Where Helix Affects the Property Model

Live Helix rejected array-valued properties in graph mutations. That directly affects the backend writer:

```
fn helix_http_properties(node: &GraphNode) -> Vec<serde_json::Value> {
    node.props
        .iter()
        .filter_map(|(key, value)| match (key.as_str(), value) {
            ("labels", _) => None,
            (_, GraphValue::String(value)) => Some(json!([key, {"Value": {"String": value}}])),
            (_, GraphValue::StringArray(_)) => None,
        })
        .collect()
}
```

This code says:

- never send the synthetic `labels` property
- send string properties
- omit string arrays

That is backend-specific behavior hidden behind the loader. FalkorDB and SurrealDB can preserve arrays; Helix currently cannot through this mutation path.

The portable export keeps string alternatives such as `tags_json` and `tags_csv` for this reason.

21. Batching With Slices

Backends use batch size like this:

```
for chunk in graph.nodes.chunks(config.batch_size.max(1)) {
    post_helix(&client, &config.helix_url, &helix_add_nodes_request(chunk)?)?;
}
```

`chunks` returns borrowed slices:

```
&[GraphNode]
```

No nodes are copied. The batch function only sees a borrowed window over the existing vector.

`config.batch_size.max(1)` prevents a panic. A chunk size of zero is invalid, so the loader clamps it to at least one.

22. Sync and Async Together

Some SDKs are async. The public loader function is synchronous:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()>
```

The SurrealDB SDK backend bridges the two with a Tokio runtime:

```
impl GraphLoader for SurrealRustSdkLoader {
    fn load(&self, config: &GraphLoadConfig, graph: &GraphData) -> Result<()> {
        let runtime = tokio::runtime::Runtime::new()
            .context("failed to create Tokio runtime")?;
        runtime.block_on(load_surrealdb_rust_sdk_async(config, graph))
    }
}
```

This is pragmatic for a command-line loader. The binary can stay simple while individual backends use async where required.

For a long-running server, you would probably make the public API async instead of creating runtimes internally.

23. Lifetimes Without Writing Lifetimes

Most of this project does not write explicit lifetime parameters. Rust infers them.

For example:

```
fn insert_optional(props: &mut BTreeMap<String, GraphValue>, key: &str, value: Option<&str>)
```

The references only need to live for the duration of the function call.

Another example:

```
fn id(&self) -> Result<&str>
```

Rust understands that the returned `&str` is tied to `&self`.

You usually write explicit lifetimes only when a function returns a borrowed value and there is more than one possible input lifetime. This code mostly has obvious borrowing relationships, so elision keeps it readable.

24. Tests as Design Locks

The loader has tests for behavior that should not regress:

```
#[test]
fn helix_http_node_batch_omits_arrays() {
    let graph = sample_graph();
    let request = helix_add_nodes_request(&graph.nodes).unwrap();
    let properties = &request["query"]["queries"][0]["Query"]["steps"][0]["AddN"]["properties"];
    assert!(
        !properties
            .as_array()
            .unwrap()
            .iter()
            .any(|prop| prop[0] == "tags")
    );
}
```

This test is not just checking syntax. It captures a live backend constraint: Helix should not receive array properties.

Another test locks the alternate importer behavior:

```
#[test]
fn converts_talk_records_to_backend_neutral_graph() {
    // Builds a TalkRecord directly and checks the resulting GraphData.
}
```

Good tests describe intent. They make future refactoring safer.

25. Practical Borrow Checker Rules From This Project

Here are the borrow checker lessons this loader teaches.

Prefer borrowing large inputs:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()>
```

Return owned output when transforming formats:

```
fn read_export_graph(input: &PathBuf) -> Result<GraphData>
```

Clone at boundaries when a new owner is needed:

```
("name", person.name.clone().into())
```

Take mutable references for helper functions that update a structure:

```
insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
```

Move values when consuming parsed input:

```
fn json_graph_value(value: serde_json::Value) -> GraphValue
```

Use slices for batches:

```
fn helix_add_nodes_request(nodes: &[GraphNode]) -> Result<serde_json::Value>
```

26. How the Abstractions Were Chosen

The main abstraction is the neutral graph:

```
GraphData
GraphNode
GraphEdge
GraphValue
```

That abstraction exists because the scraper should not know how FalkorDB, HelixDB, or SurrealDB write graph data.

The second abstraction is backend selection:

```
GraphBackend
GraphLoadConfig
load_graph(...)
```

That abstraction exists because the CLI needs to select a backend at runtime.

The third abstraction is internal:

```
trait GraphLoader
```

That abstraction exists because backend implementations should have a common shape while keeping the public API stable.

A useful rule: abstractions should protect a real boundary. Here the real boundaries are:

- input format versus graph model
- graph model versus database writes
- public API versus private backend implementation

27. What Could Improve Next

The current implementation is intentionally practical. Good next steps would be:

- add automated live integration tests for all three databases
- add backend-specific read helpers for common graph traversals
- support typed numeric and date properties in `GraphValue`
- add backend upsert modes for Helix and SurrealDB where available
- split backend modules into separate files once the single module becomes too large

Do not split files just to split files. Split when navigation, ownership, or testability improves.

Conclusion

This graph loader is a useful Rust learning project because it is small enough to understand and real enough to show the language's strengths.

It demonstrates:

- typed command-line parsing
- owned data models
- borrowed public APIs
- fallible conversion with `Result`
- structured backend dispatch
- careful string generation
- importer normalization
- tests that encode backend constraints

The most important Rust idea here is ownership at boundaries. Parse into owned data. Borrow while loading. Clone only when a second owner is actually needed. Move values when consuming an input format. Keep backend-specific details behind a stable interface.

That is the heart of writing clear Rust.

28. Edition 2: The Whole Project Architecture

The full project is more than the loader. It is a data pipeline with three architectural zones:

1. acquisition and extraction in `src/main.rs`
2. import-shape normalization in `src/bin/load_graph.rs`
3. backend-neutral graph loading in `src/graph_loader.rs`

The executable in `src/main.rs` owns the messy edge of the world. It knows about URLs, HTML, Meetup GraphQL, LLM extraction, hand-written parsers, raw source records, and the final consolidated export. The loader binary owns the contract between exported data and the portable graph model. The library module owns the contract between the portable graph model and databases.

That split is the main architectural decision of the codebase. Unreliable input formats and backend-specific write protocols are not allowed to leak into each other. The scraper can improve its parsing rules without caring whether the graph is later written to FalkorDB, HelixDB, or SurrealDB. The database loader can add a backend without knowing how Meetup pages were scraped.

The project is arranged as a pipeline:

```
remote pages and APIs
    -> raw source records
    -> extracted structured records
    -> GraphExport JSON
    -> GraphData
    -> backend-specific writes
```

Each stage consumes data that has become stable enough for that layer, then emits a simpler value for the next layer.

29. Crate Layout and Module Boundaries

The crate is named `bythebay-scraper`, but it exposes a reusable library module:

```
pub mod graph_loader;
```

That one-line `src/lib.rs` is small, but it changes the architecture. It lets the `load_graph` binary use the loader through the crate boundary:

```
use bythebay_scraper::graph_loader::{
    GraphBackend, GraphData, GraphEdge, GraphLoadConfig, GraphNode, GraphValue, load_graph,
};
```

This avoids copying loader code into the binary. It also makes public and private API explicit. `GraphData`, `GraphNode`, `GraphEdge`, `GraphValue`, `GraphBackend`, `GraphLoadConfig`, and `load_graph` are public because they are the loader contract. The individual backend structs and helper functions are private because callers should not depend on their details.

The module boundary is deliberately thin:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()>
```

Everything behind it can change as long as this contract remains stable.

30. The Two Executables

The default binary in `src/main.rs` is the scraper/exporter. It uses Tokio and `reqwest::Client`, so its `main` function is asynchronous. Its job is broad: it fetches conference pages, fetches Meetup archives, chooses between LLM extraction and local parsing, writes split records under `data/raw`, builds `GraphExport`, and writes `data/bythebay-graph.json`.

The loader binary in `src/bin/load_graph.rs` is narrower:

```
fn main() -> Result<()> {
    let args = Args::parse();
    let graph = read_graph_data(&args.input, args.input_format)?;
    load_graph(&GraphLoadConfig::from(&args), &graph)
}
```

That short `main` is a strong signal. The binary parses arguments, reads input, and delegates. The interesting loader behavior lives in the library, not in the executable wrapper.

31. Data Contracts: Raw, Export, Graph

The project uses several data shapes because one shape would be too vague:

- raw source records preserve what was fetched
- extracted records preserve what was understood from a source
- `GraphExport` is a stable interchange file
- `GraphData` is the database-neutral loader model

In `src/main.rs`, structs such as `RawSpeaker`, `RawTalk`, `RawMeetup`, `RawMeetupEvent`, and `RawMeetupSession` reflect scraper concerns. They contain optional fields because real web data is incomplete.

In `src/bin/load_graph.rs`, `GraphExport` is closer to the graph:

```
struct GraphExport {
    source_urls: Vec<String>,
    conferences: Vec<Conference>,
    meetups: Vec<MeetupGroup>,
    people: Vec<Person>,
    talks: Vec<Talk>,
    edges: Vec<Edge>,
}
```

In `src/graph_loader.rs`, `GraphData` removes domain-specific record names:

```
pub struct GraphData {
    pub nodes: Vec<GraphNode>,
    pub edges: Vec<GraphEdge>,
}
```

Earlier stages are rich in domain vocabulary. Later stages are rich in graph vocabulary.

32. `serde`: Deriving the Boundary

The codebase uses `serde` to turn external JSON into typed Rust values:

```
#[derive(Debug, Deserialize)]
struct TalkRecordNode {
    id: String,
    #[serde(alias = "type", alias = "kind")]
    label: String,
    #[serde(default)]
    properties: serde_json::Map<String, serde_json::Value>,
}
```

`derive` asks the compiler to generate trait implementations. `serde(alias = "...")` accepts alternate input field names. `serde(default)` makes missing fields explicit. The project uses dynamic `serde_json::Value` at the uncertain edge, then converts into typed structs when the shape is known.

33. Enums as Closed Worlds

The project uses enums where a value should be one of a known set:

```
pub enum GraphBackend {
    Falkor,
    HelixHttp,
    HelixRustSdk,
    Surrealdb,
    SurrealdbRustSdk,
}
```

This gives the compiler a complete list of backend choices. If a new backend is added, the match in `load_graph` points at every dispatch site that must be updated.

`GraphValue` is another closed world:

```
pub enum GraphValue {
    String(String),
    StringArray(Vec<String>),
}
```

The loader intentionally supports only values that all target backends can handle predictably. The enum is a portable property contract.

34. Traits as Backend Ports

The loader uses one private trait:

```
trait GraphLoader {
    fn load(&self, config: &GraphLoadConfig, graph: &GraphData) -> Result<()>;
}
```

Each backend implements the same operation:

```
impl GraphLoader for FalkorLoader { /* ... */ }
impl GraphLoader for HelixHttpLoader { /* ... */ }
impl GraphLoader for HelixRustSdkLoader { /* ... */ }
impl GraphLoader for SurrealHttpLoader { /* ... */ }
impl GraphLoader for SurrealRustSdkLoader { /* ... */ }
```

This is a port-and-adapter design in compact Rust form. The trait names the port: “load this graph.” The structs are adapters: “load it into Falkor,” “load it into Helix through HTTP,” “load it into SurrealDB through the SDK.”

The trait is private because there is no need for callers to implement their own loaders yet. Keeping it private gives the project freedom to change the backend internals without breaking the public API.

35. SDK Backends Versus HTTP Backends

HelixDB and SurrealDB each have two loader paths.

The HTTP variants build request payloads or query strings directly, then post them through `reqwest::blocking::Client`.

The Rust SDK variants use client libraries and SDK-specific builders, then run async operations inside a Tokio runtime created by the synchronous loader.

For HelixDB, the HTTP path builds JSON manually:

```
json!({
    "request_type": "write",
    "query": {"queries": queries, "returns": returns},
    "parameters": {},
    "parameter_types": {}
})
```

The Helix SDK path builds a dynamic query with typed DSL calls:

```
let request = DynamicQueryRequest::write(batch.returning(returns));
client.query().dynamic_query(request).send().await?;
```

The HTTP path is transparent because you can inspect the exact JSON payload. The SDK path is more structured because Rust types and builder methods catch some mistakes before the request is sent.

For SurrealDB, the HTTP path posts SurrealQL to `/sql` with headers for namespace and database. The SDK path connects over WebSocket:

```
let address = surreal_ws_address(&config.surreal_url)?;
let db = Surreal::new::<Ws>(&address).await?;
```

That means the same user-facing `--surreal-url http://127.0.0.1:8000/sql` is adapted for the SDK. The helper `surreal_ws_address` exists because the SDK and HTTP interfaces do not use identical endpoint shapes.

36. What Got Reused and Abstracted

The SDK loaders do not duplicate the whole backend implementation. They reuse the same neutral graph model, config, batching policy, and transformation helpers.

Helix HTTP and Helix SDK both use `relationship_type`, `helix_base_url`, graph chunks from `graph.nodes.chunks(config.batch_size.max(1))`, and the same omission of `labels` and string arrays from Helix properties. They differ in only the final representation: HTTP builds `serde_json::Value`; the SDK builds `write_batch()` DSL requests.

SurrealDB HTTP and SurrealDB SDK both use `surreal_bootstrap_query`, `surreal_delete_tables_query`, `surreal_create_nodes_query`, `surreal_create_edges_query`, `surreal_id_tables`, `surreal_table_name`, and `surreal_string`. They differ in transport and session management: HTTP sends Basic auth and namespace/database headers per request; the SDK signs in once, selects namespace/database, and sends queries through the SDK.

The code abstracts backend choice, graph property values, repeated query construction, and repeated data-cleaning operations. It reuses data contracts and semantic helpers first. It abstracts execution strategies only when they really have the same shape.

37. Functional Programming in the Codebase

The project uses a practical Rust version of functional programming: iterators, maps, filters, folds-by-collection, and expression-oriented transforms.

Consider `source_nodes`:

```
source_urls
    .iter()
    .filter(|url| seen.insert((*url).clone()))
    .map(|url| GraphNode::new(/* ... */))
    .collect()
```

This pipeline says: borrow each URL, keep only the first occurrence, transform it into a `GraphNode`, and collect the result.

Consider the JSON property conversion:

```
values
    .into_iter()
    .filter_map(|value| match value {
        serde_json::Value::String(value) if !value.is_empty() => Some(value),
        _ => None,
    })
    .collect()
```

`filter_map` combines validation and transformation. Invalid array entries disappear. Valid strings move into the output vector.

Rust's iterator style is not merely cosmetic. It controls ownership. `iter()` borrows. `into_iter()` consumes. `cloned()` makes owned copies at visible points.

38. Option as a Data-Cleaning Tool

The project uses `Option<T>` for missing data, but also as a small functional pipeline:

```
fn insert_optional(props: &mut BTreeMap<String, GraphValue>, key: &str, value: Option<&str>) {
    if let Some(value) = value.filter(|value| !value.is_empty()) {
        props.insert(key.to_string(), value.into());
    }
}
```

This helper reuses one rule everywhere: only insert optional string properties when they exist and are not empty. The function takes `Option<&str>`, not `Option<String>`, because callers often hold borrowed data:

```
insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
```

The scraper uses `Option` similarly:

```
title.map(clean_text).and_then(nonempty)
```

`map` transforms the value if it exists. `and_then` chains a transformation that may itself reject the value.

39. Result and Error Context

The project uses `anyhow::Result` for application-level errors. This is appropriate because the binaries need clear failure messages more than they need a stable typed error API.

The code often adds context at I/O boundaries:

```
fs::read_to_string(input)
    .with_context(|| format!("failed to read {}", input.display()))?;
```

The `?` operator returns early if the operation failed. `with_context` preserves the original error and adds local meaning. The loader also uses `bail!` when the program detects a semantic problem, such as a graph node without a string `id`.

40. Ownership: Moves, Clones, and Reuse

The project clones in places where duplicated ownership is simpler than lifetime coupling:

```
redis_url: args.redis_url.clone()
```

`Args` and `GraphLoadConfig` both own their strings. That makes the config easy to pass around without borrowing from the CLI parser's local value.

The project moves values when input ownership is no longer needed:

```
nodes_by_id.into_values().collect()
```

After deduplicating nodes in a map, the map itself is no longer useful. `into_values` consumes it and moves each `GraphNode` into the output vector.

The project borrows when it only needs to inspect:

```
for node in &graph.nodes {
    // read node and write backend query
}
```

41. Small Helper Functions as Policy Reuse

The most successful reuse in the project is small and direct.

`props` removes repeated map boilerplate. `insert_optional` centralizes optional-property filtering. `relationship_type` centralizes backend-safe relationship labels. `surreal_string` delegates string escaping to `serde_json::to_string`, avoiding hand-written quote escaping.

`surreal_table_name`, `surreal_identifier`, and `relationship_type` centralize name normalization rules so every backend query does not invent its own rules. `post_helix`, `post_surrealdb`, and `post_surrealdb_sdk` centralize response checking. This is not just code reuse. It is policy reuse: what counts as a failed backend write lives in one helper per transport.

42. Async Rust in a Mostly Synchronous Loader

The scraper is naturally async because it fetches remote resources. It uses Tokio and async `reqwest`:

```
async fn fetch(client: &Client, url: &str) -> Result<String>
```

The loader binary is mostly synchronous because file reading, command-line parsing, and Falkor/HTTP loading are simple blocking operations.

The SDK backends introduce async APIs into that synchronous loader. The code bridges the two worlds by creating a runtime:

```
let runtime = tokio::runtime::Runtime::new()?;
runtime.block_on(post_helix_sdk_nodes(&client, chunk))?;
```

This keeps the public `GraphLoader` trait synchronous. If the project later needs high-concurrency loading, it could make the trait async or split sync and async loaders. Today, the synchronous trait is simpler.

43. Generics in the Project

The codebase uses generics where they pay their rent.

The `props` helper uses const generics:

```
fn props<const N: usize>(items: [(&str, GraphValue); N]) -> BTreeMap<String, GraphValue>
```

This lets callers pass arrays of any fixed length without heap-allocating a temporary vector.

The scraper uses a type parameter for JSON writing:

```
fn write_source_json<T: Serialize>(raw_dir: &Path, file_name: &str, value: &T) -> Result<()>
```

The LLM extractor uses a higher-ranked trait bound:

```
async fn extract_json<T: for<'de> Deserialize<'de>>(...)
```

The practical meaning is: “give me any type that serde can deserialize from this response text.”

SurrealDB SDK helpers are generic over connection type:

```
async fn post_surrealdb_sdk<C>(db: &Surreal<C>, query: &str) -> Result<()>
where
    C: surrealdb::Connection,
```

44. Pattern Matching and `let else`

Rust pattern matching appears everywhere in the project. The most visible examples are `match` expressions over enums and JSON values.

The scraper also uses `let else` for early exits inside parsing logic:

```
let Some(speaker_text) = speakers.get(&id).map(|s| clean_text(s)) else {
    continue;
};
```

That reads as: if this optional value exists, bind it; otherwise continue the loop. It is clearer than a nested `if let` when the rest of the loop requires the value.

45. Collections and Determinism

The project often uses `BTreeMap` and `BTreeSet` instead of hash-based collections. That decision gives deterministic output order and combines deduplication with sorting.

For talk-record import, this key deduplicates edges by source, target, and relationship:

```
BTreeMap<(String, String, String), GraphEdge>
```

Determinism matters for generated files, test expectations, and database query debugging. A graph loader that emits stable output is easier to reason about.

46. Backend-Specific Semantics

FalkorDB accepts Cypher-like strings through Redis commands. Its loader creates one query per node and one query per edge. Labels are encoded directly in the Cypher pattern, and properties are written with `SET n +=`.

HelixDB HTTP accepts a dynamic-query JSON format. Nodes and edges become a list of named query steps plus named returns.

HelixDB SDK accepts typed DSL expressions. The code builds a write batch, names intermediate variables with `var_as`, and returns selected variables.

SurrealDB accepts SurrealQL. Nodes are created with table/id pairs, and relationships are created with `RELATE`.

These databases do not share one query language. The architecture shares the portable graph and isolates each query language behind a backend adapter.

47. Property Model Differences Across Backends

The property model is where backend differences become visible.

FalkorDB can receive strings and arrays in generated Cypher, so `cypher_map` serializes both `GraphValue::String` and `GraphValue::StringArray`.

SurrealDB can receive strings and arrays in SurrealQL assignments, so `surreal_node_props` writes both.

Helix currently receives only string properties from this loader. Both Helix paths omit string arrays:

```
(_, GraphValue::StringArray(_)) => None
```

That is why the export includes multiple tag representations: `tags` as `StringArray`, `tags_csv` as a string, and `tags_json` as a string. Backends with array support can use `tags`. Backends with string-only property support still receive useful tag data.

48. Testing as Executable Design Notes

The tests document decisions that are easy to break accidentally.

The loader tests assert that Helix omits arrays, edge batches use target variables correctly, SurrealDB creates complete sample graph statements, replace queries include known labels, and URL conversion works for SDK endpoints.

The import tests assert that export records and talk-record files both become the same neutral `GraphData` shape.

These tests are design locks. They protect the small transformations that would be painful to debug only after a database write failed.

49. Key Rust Features Referenced by the Codebase

This project touches a wide set of practical Rust features:

- `struct` for domain records and configuration
- `enum` for closed choices and portable value types
- `trait` for backend adapter behavior
- `impl` blocks for constructors, conversions, and backend implementations
- `derive` macros for `Debug`, `Clone`, `Default`, `Deserialize`, `Serialize`, and `ValueEnum`
- attributes such as `#[arg(...)]`, `#[serde(...)]`, `#[tokio::main]`, and `#[cfg(test)]`
- owned types such as `String`, `PathBuf`, `Vec<T>`, `BTreeMap<K, V>`, and `BTreeSet<T>`
- borrowed types such as `&str`, `&PathBuf`, `&GraphData`, and slices like `&[GraphNode]`
- `Option<T>` for missing data
- `Result<T>` and `?` for fallible code
- iterators: `iter`, `into_iter`, `map`, `filter`, `filter_map`, `flat_map`, `collect`
- closures for inline transformations
- generics and trait bounds
- const generics
- async functions and `.await`
- pattern matching, tuple keys, and `let else`
- modules and crate-level reuse
- tests with `#[test]`

The valuable point is not that the project uses many features. The valuable point is that each feature has a job. Rust is strongest when types, ownership, and module boundaries describe the architecture directly.

50. Edition 2 Summary

The first edition used the loader to explain the borrow checker. Edition 2 shows the wider design:

- scrape and extraction code stays at the edge
- exported data becomes a stable contract
- loader input normalizes into `GraphData`
- backend adapters implement one private trait
- HTTP and SDK paths share semantics but differ in transport
- functional iterator style keeps transformations compact
- small helpers reuse policy without over-abstracting the system

The result is not a giant framework. It is a practical Rust application whose architecture is visible in its types.